

Practical 6

AIM: Implementation of chain matrix multiplication using dynamic programming.

1 ALGORITHM:

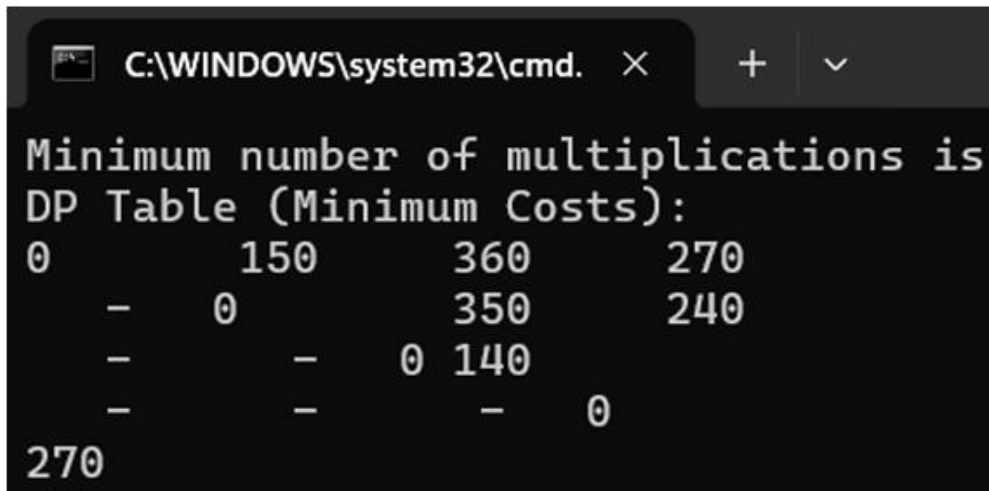
Algorithm:-
Input:- number of matrices 'n'.
 * dimensions of each matrix in the form of an array PCJ.
 * If there are n matrices, PCJ contains n+1 dimensions.
Output:- Display the minimum of scalar multiplication required.
Step1:- Start.
Step2:- Read the number of matrices 'n'.
Step3:- Store dimensions of matrices in array PCJ.
Step4:- Create a table $M[i][j]$ where,
 $M[i][j]$ as = minimum cost of multiply A; through all.
Step5:- Consider chains of length 2, 3, ... n.
Step6:- For any chain $A_i \dots A_j$ for every function $SAC(i, k, j)$.
 $(A_i \dots A_k) \times (A_{k+1} \dots A_j)$.
 calculate $cost = M[i][k] + M[k+1][j] + p[i-1] * p[k] * p[j]$.
Step7:- Store minimum cost in $M[i][j]$.
Step8:- Finally $m(i, n)$ gives the minimum no. of multiplication.

2 PROGRAM:

```
#include <iostream>
#include <climits>
using namespace std;
void printTable(int m[][100], int n) {
    cout << "\nDP Table (Minimum Costs):\n";
    for (int i = 1; i < n; i++) {
        for (int j = 1; j < n; j++) {
            if (i > j) cout << "  -  ";
            else cout << m[i][j] << "\t";
        }
        cout << endl;
    }
}
int matrixChainOrder(int p[], int n) {
    int m[100][100];
    for (int i = 1; i < n; i++)
        m[i][i] = 0;
```

```
for (int L = 2; L < n; L++) {
    for (int i = 1; i < n - L + 1; i++) {
        int j = i + L - 1;
        m[i][j] = INT_MAX;
        for (int k = i; k < j; k++) {
            int q = m[i][k] + m[k+1][j] + p[i-1]*p[k]*p[j];
            if (q < m[i][j])
                m[i][j] = q;
        }
    }
}
printTable(m, n);
return m[1][n-1];}
int main() {
    int arr[] = {3, 5, 10, 7, 2};
    int n = sizeof(arr)/sizeof(arr[0]);
    cout << "Minimum number of multiplications is "
        << matrixChainOrder(arr, n) << endl;
    return 0;
}
```

3 OUTPUT:



```
C:\WINDOWS\system32\cmd.  X  +  v

Minimum number of multiplications is
DP Table (Minimum Costs):
0      150      360      270
-      0      350      240
-      -      0 140
-      -      -      0
270
```

4 TIME COMPLEXITY:

Time complexity:-
for len=2
for i=1 to n-length-1 $\rightarrow$ n times.
for i=1 to n-length-1 $\rightarrow$ n times
for k=i to j-1 $\rightarrow$ n times
So approximates $n \times n \times n = n^3$,
check time complexity $O(n^3) \rightarrow$ best case
op 1 $\rightarrow$ length = 2 to n $\rightarrow$ n
op 2 $\rightarrow$ 1 to n-length+1 $\rightarrow$ n
op 3 $\rightarrow$ k=i to j-1 $\rightarrow$ n
no. of operations $= n \times n \times n = n^3$.
∴ time complexity is same for all cases.
Best case = worst case = average case = n^3 .

5 Conclusion:

Matrix Chain Multiplication using Dynamic Programming efficiently finds the minimum cost of multiplying a chain of matrices by solving subproblems optimally.